

CalSync.ai — Database Schema Reference

Team ByteMinds · INSPIRON 5.0 · Problem Statement #3

How to use this document: This is the single source of truth for the Supabase database schema. Every developer — backend, frontend, and agent — should refer to this before writing any query, type definition, or API contract. If the schema changes, update this file first and notify the team.

Quick Navigation

#	Section	Tables
0	Enums	All shared types
1	Auth & Profiles	profiles
2	OAuth Tokens	oauth_tokens
3	Gmail Watch	gmail_watch_subscriptions
4	Participant Preferences	participant_preferences
5	Sessions	sessions
6	Session Participants	session_participants
7	Emails	emails
8	Email Drafts	email_drafts
9	Booked Meetings	booked_meetings
10	Meeting Attendees	meeting_attendees
11	Session Overrides	session_overrides
12	Activity Logs	activity_logs
13	Idempotency	processed_email_hashes

14	App Settings	<code>app_settings</code>
15	Views	6 pre-built views
16	Functions	Helper functions
17	Who Uses What	Per-role reference

0. Enums

All enum types used across tables. **Never use raw strings for these — always reference the enum value.**

`session_status`

Controls the state machine for a scheduling session.

Value	Meaning
<code>AWAITING_REPLIES</code>	Session created, waiting for participants to submit availability
<code>READY_TO_COMPUTE</code>	All participants have replied, ready for overlap computation
<code>COMPUTING</code>	Agent is currently computing overlap (brief transient state)
<code>BOOKED</code>	Meeting successfully booked on Google Calendar
<code>NO_OVERLAP</code>	Overlap computed but no common slot found — needs clarification
<code>EXPIRED</code>	Session TTL (7 days) exceeded without resolution
<code>CANCELLED</code>	Manually cancelled via human override

State machine flow:

```

AWAITING_REPLIES → READY_TO_COMPUTE → COMPUTING → BOOKED
                    ↘ NO_OVERLAP
                      ↘ EXPIRED (after 7 days, any unresolved status)
                        ↘ CANCELLED (human override, any status)
  
```

`email_processing_status`

Tracks the pipeline state of each inbound email.

Value	Meaning
PENDING	Email received and stored, not yet processed by agent
PROCESSING	Agent is currently handling this email
DONE	Agent finished processing successfully
FAILED	Processing failed — see <code>processing_error</code> column
DUPLICATE	SHA-256 hash matched an existing email — silently discarded
IGNORED	Not scheduling-related (failed keyword check) — no action taken

override_action

Type of human override applied to a session.

Value	Meaning
CANCEL	Cancel the session and delete the calendar event
RESCHEDULE	Cancel existing event, book a new one at specified time
FORCE_BOOK	Book a specific slot regardless of availability check

log_event_type

Category of each entry in the activity log.

Value	Triggered By
EMAIL_RECEIVED	New email ingested from Gmail
EMAIL_PROCESSED	Email processing completed
EMAIL_SENT	Agent sent an outbound email
SESSION_CREATED	New scheduling session started

SLOT_UPDATED	Participant submitted their availability
OVERLAP_COMPUTED	Overlap algorithm ran (success or no-overlap)
MEETING_BOOKED	Google Calendar event successfully created
MEETING_CANCELLED	Event cancelled (agent or human)
MEETING_RESCHEDULED	Event moved to new time
OVERRIDE_APPLIED	Human override action executed
CLARIFICATION_SENT	Agent sent follow-up for ambiguous availability
THREAD_SUMMARISED	Agent generated a thread summary
PREFERENCE_UPDATED	Participant preferences changed
AUTH_REFRESHED	Google OAuth token refreshed
WATCH_RENEWED	Gmail watch() subscription renewed
ERROR	Any unhandled error in the pipeline

log_severity

Value	Colour in UI	Used For
INFO	Grey	Normal operations
SUCCESS	Green	Positive outcomes (booked, sent)
WARNING	Amber	Non-blocking issues (no overlap, expiring watch)
ERROR	Red	Failures requiring attention

day_of_week

MON · TUE · WED · THU · FRI · SAT · SUN

draft_status

Value	Meaning
PENDING_REVIEW	Draft awaiting human approval
APPROVED	Approved, queued to send
SENT	Successfully sent via Gmail API
DISCARDED	Rejected by human reviewer

attendee_rsvp_status

Mirrors Google Calendar API RSVP values exactly.

Value	Meaning
needsAction	Invite sent, no response yet
declined	Attendee declined
tentative	Attendee accepted tentatively
accepted	Attendee accepted

1. Auth & Profiles

public.profiles

Extends Supabase's built-in `auth.users` table. One row per operator who logs into the dashboard.

Column	Type	Nullable	Default	Description
--------	------	----------	---------	-------------

<code>id</code>	<code>uuid</code>	NO	—	Primary key. Foreign key → <code>auth.users(id)</code>
<code>email</code>	<code>text</code>	NO	—	Login email address
<code>display_name</code>	<code>text</code>	YES	—	Shown in top bar of dashboard
<code>avatar_url</code>	<code>text</code>	YES	—	Profile picture URL
<code>google_email</code>	<code>text</code>	YES	—	Gmail address used as CalSync inbox (may differ from login)
<code>google_connected</code>	<code>boolean</code>	NO	<code>false</code>	Whether Google OAuth is set up
<code>display_timezone</code>	<code>text</code>	NO	<code>'UTC'</code>	IANA timezone for dashboard time display
<code>created_at</code>	<code>timestamp tz</code>	NO	<code>NOW()</code>	—
<code>updated_at</code>	<code>timestamp tz</code>	NO	<code>NOW()</code>	Auto-updated on change

Notes for frontend: Query `profiles` on login to get `display_timezone` for time formatting across the app. Show `google_connected` status on the Settings page.

Notes for backend: Created automatically via Supabase Auth trigger. Do not manually insert.

2. OAuth Tokens

`public.oauth_tokens`

Stores Google OAuth refresh tokens. One row per connected Google account.

Column	Type	Nullable	Default	Description
<code>id</code>	<code>uuid</code>	NO	<code>uuid_generate_v4()</code>	Primary key
<code>profile_id</code>	<code>uuid</code>	NO	—	FK → <code>profiles(id)</code>

<code>provider</code>	<code>text</code>	NO	<code>'google'</code>	OAuth provider name
<code>refresh_token</code>	<code>text</code>	NO	—	Long-lived refresh token from Google
<code>client_id</code>	<code>text</code>	NO	—	Google OAuth client ID
<code>client_secret</code>	<code>text</code>	NO	—	Google OAuth client secret
<code>scopes</code>	<code>text[]</code>	NO	<code>'{'}</code>	Granted OAuth scopes
<code>token_expires_at</code>	<code>timestamp tz</code>	YES	—	When last access token expires (informational)
<code>created_at</code>	<code>timestamp tz</code>	NO	<code>NOW()</code>	—
<code>updated_at</code>	<code>timestamp tz</code>	NO	<code>NOW()</code>	—

Unique constraint: (`profile_id`, `provider`) — one token set per provider per user.

Notes for backend: Read `refresh_token`, `client_id`, `client_secret` from this table at runtime. Use them to generate short-lived access tokens via Google's token endpoint. Never store access tokens — always derive them.

Notes for frontend: Only read `google_connected` from `profiles`. Never read `oauth_tokens` directly from the frontend (RLS blocks it for non-owners).

3. Gmail Watch Subscriptions

`public.gmail_watch_subscriptions`

Tracks the active Gmail push notification subscription. Must be renewed every 7 days.

Column	Type	Nullabl e	Default	Description
<code>id</code>	<code>uuid</code>	NO	<code>uuid_generate_v4()</code>	Primary key
<code>profile_id</code>	<code>uuid</code>	NO	—	FK → <code>profiles(id)</code>

<code>gmail_addresses</code>	<code>text</code>	NO	—	Gmail address being watched
<code>pubsub_topic</code>	<code>text</code>	NO	—	Google Cloud Pub/Sub topic name
<code>history_id</code>	<code>text</code>	YES	—	Gmail history ID from last <code>watch()</code> call
<code>expires_at</code>	<code>timestamp tz</code>	NO	—	When this subscription expires (max 7 days)
<code>is_active</code>	<code>boolean</code>	NO	<code>true</code>	Whether subscription is currently active
<code>last_renewed_at</code>	<code>timestamp tz</code>	NO	<code>NOW()</code>	Last renewal timestamp
<code>created_at</code>	<code>timestamp tz</code>	NO	<code>NOW()</code>	—

Notes for frontend: Display `expires_at` and `is_active` on the Settings page. Show a warning banner if `expires_at < NOW() + 2 days`. The "Renew Gmail Watch" button calls the backend which updates this row.

Notes for backend: A `pg_cron` job runs daily and logs a `WARNING` to `activity_logs` if expiry is within 2 days. The Gmail MCP calls `watch()` and upserts this row on renewal.

4. Participant Preferences

`public.participant_preferences`

Per-email-address scheduling preferences. Not tied to auth users — any email can have preferences set.

Column	Type	Nullable	Default	Description
<code>id</code>	<code>uuid</code>	NO	<code>uuid_generate_v4()</code>	Primary key
<code>email</code>	<code>text</code>	NO	—	Unique. Participant's email address

timezone	text	NO	'UTC'	IANA timezone e.g. 'Asia/Kolkata'
working_hours_start	time	NO	'09:00'	Start of workday in participant's timezone
working_hours_end	time	NO	'18:00'	End of workday in participant's timezone
preferred_days	day_of_week[]	NO	'{MON, TUE, WED, THU, FRI}'	Available days of week
is_vip	boolean	NO	false	VIP flag — gets 1.5x weight in overlap scoring
notes	text	YES	—	Operator notes, visible only in dashboard
created_at	timestampz	NO	NOW()	—
updated_at	timestampz	NO	NOW()	Auto-updated on change

Unique constraint: email

Notes for frontend: This entire table is managed from the Preferences page. Display `working_hours_start` and `working_hours_end` converted to the participant's `timezone`. The VIP toggle maps directly to `is_vip`.

Notes for backend: Before computing overlap, fetch preferences for all participants. If `is_vip = true`, apply 1.5x weight to that participant's available slots in the scoring algorithm.

5. Sessions

public.sessions

The core coordination unit. One row per Gmail thread. This is the most important table in the application.

Column	Type	Nullable	Default	Description
session_id	text	NO	—	Primary key. Format: <code>sess_{md5(thread_id)[:8]}</code>
thread_id	text	NO	—	Unique. Gmail thread ID
meeting_title	text	NO	'Meeting'	Human-readable title for the dashboard
organizer_email	text	NO	—	Email of the person who initiated scheduling
participants	text[]	NO	' {} '	All participant emails including organizer
status	session_status	NO	'AWAITING_REPLIES'	Current state machine status
collected_slots	jsonb	NO	' {} '	Availability slots per participant (see shape below)
overlap_results	jsonb	YES	—	Top 3 computed overlap slots (see shape below)
booked_slot	jsonb	YES	—	The slot that was booked (see shape below)
booked_event_id	text	YES	—	Google Calendar event ID

booked_event_link	text	YES	—	Google Calendar event URL
meet_link	text	YES	—	Google Meet URL
thread_summary	text	YES	—	Agent-generated plain-English thread summary
clarification_count	integer	NO	0	Number of clarification emails sent
expires_at	timestampz	NO	NOW() + 7 days	When to mark session EXPIRED
created_at	timestampz	NO	NOW()	—
updated_at	timestampz	NO	NOW()	Auto-updated on change

JSONB Field Shapes

collected_slots

```
{
  "bob@example.com": [
    { "start": "2025-01-06T08:30:00Z", "end": "2025-01-06T10:30:00Z", "confidence": 0.92 },
    { "start": "2025-01-07T03:30:00Z", "end": "2025-01-07T07:30:00Z", "confidence": 0.78 }
  ],
  "carol@example.com": [
    { "start": "2025-01-06T08:00:00Z", "end": "2025-01-06T09:30:00Z", "confidence": 0.85 }
  ]
}
```

overlap_results

```
[
  { "start": "2025-01-06T08:30:00Z", "end": "2025-01-06T09:30:00Z", "score": 1.5 },
  { "start": "2025-01-07T05:00:00Z", "end": "2025-01-07T06:00:00Z", "score": 1.0 },
  { "start": "2025-01-08T08:00:00Z", "end": "2025-01-08T09:00:00Z", "score": 0.8 }
]
```

booked_slot

```
{ "start": "2025-01-06T08:30:00Z", "end": "2025-01-06T09:30:00Z" }
```

Key rules:

- All datetime strings are **ISO 8601 UTC** — no exceptions
 - `collected_slots` keys are participant email addresses
 - `confidence` is a float 0.0–1.0 from the NLP parser
 - `score` in `overlap_results` is weighted by VIP priority
-

6. Session Participants

`public.session_participants`

Normalised per-participant reply tracking. Auto-populated from `sessions.participants` array on session INSERT via trigger.

Column	Type	Nullable	Default	Description
<code>id</code>	<code>uuid</code>	NO	<code>uuid_generate_v4()</code>	Primary key
<code>session_id</code>	<code>text</code>	NO	—	FK → <code>sessions(session_id)</code> CASCADE DELETE
<code>participant_email</code>	<code>text</code>	NO	—	Participant email address
<code>has_replied</code>	<code>boolean</code>	NO	<code>false</code>	Whether this participant submitted availability
<code>replied_at</code>	<code>timestamp tz</code>	YES	—	When they replied (<code>null</code> if still pending)
<code>reminder_count</code>	<code>integer</code>	NO	<code>0</code>	Number of reminder emails sent
<code>last_reminder_at</code>	<code>timestamp tz</code>	YES	—	When last reminder was sent

<code>created_at</code>	<code>timestamp tz</code>	NO	<code>NOW()</code>	—
<code>updated_at</code>	<code>timestamp tz</code>	NO	<code>NOW()</code>	Auto-updated on change

Unique constraint: (`session_id`, `participant_email`)

Notes for frontend: Use this table (via `v_sessions_summary`) to display the replied/pending participant lists in `SessionCard` and `Session Detail`. Do not try to derive this from `sessions.collected_slots` manually.

Notes for backend: When calling `update_slot`, also set `has_replied = true` and `replied_at = NOW()` for that participant in this table.

7. Emails

`public.emails`

Every email processed by CalSync — both inbound from participants and outbound sent by the agent.

Column	Type	Nullable	Default	Description
<code>id</code>	<code>uuid</code>	NO	<code>uuid_generate_v4()</code>	Primary key
<code>message_id</code>	<code>text</code>	NO	—	Unique. Gmail message ID
<code>thread_id</code>	<code>text</code>	NO	—	Gmail thread ID (groups related messages)
<code>session_id</code>	<code>text</code>	YES	—	FK → <code>sessions(session_id)</code> SET NULL on delete
<code>direction</code>	<code>text</code>	NO	<code>'INBOUND'</code>	<code>'INBOUND'</code> or <code>'OUTBOUND'</code>

<code>from_email</code>	<code>text</code>	NO	—	Sender email address
<code>to_emails</code>	<code>text[]</code>	NO	<code>'{'}</code>	Recipient email addresses
<code>cc_emails</code>	<code>text[]</code>	NO	<code>'{'}</code>	CC email addresses
<code>subject</code>	<code>text</code>	NO	<code>''</code>	Email subject line
<code>body_text</code>	<code>text</code>	NO	<code>''</code>	Plain-text email body (always populated)
<code>body_html</code>	<code>text</code>	YES	—	HTML email body (when available)
<code>labels</code>	<code>text[]</code>	NO	<code>'{'}</code>	Gmail labels applied by agent
<code>processing_status</code>	<code>email_processing_status</code>	NO	<code>'PENDING'</code>	Pipeline status
<code>processing_error</code>	<code>text</code>	YES	—	Error message if <code>status = 'FAILED'</code>
<code>email_hash</code>	<code>text</code>	NO	—	Unique. SHA-256 of <code>(from_email + ":" + message_id)</code>
<code>is_read</code>	<code>boolean</code>	NO	<code>false</code>	Whether marked as read in Gmail
<code>has_attachments</code>	<code>boolean</code>	NO	<code>false</code>	Whether email has file attachments
<code>received_at</code>	<code>timestampz</code>	NO	<code>NOW()</code>	When Gmail received the email
<code>ingested_at</code>	<code>timestampz</code>	NO	<code>NOW()</code>	When CalSync stored it
<code>processed_at</code>	<code>timestampz</code>	YES	—	When processing completed
<code>created_at</code>	<code>timestampz</code>	NO	<code>NOW()</code>	—

Notes for frontend: Use `v_email_feed` view (not this table directly) — it includes the associated session's `meeting_title` and `session_status`. Show `body_text` truncated to 160 chars in the list, full in the detail panel.

Notes for backend: Always populate `email_hash` before inserting. Check `processed_email_hashes` table (or call `fn_check_and_mark_email()`) before setting `processing_status = 'PROCESSING'`.

8. Email Drafts

`public.email_drafts`

Agent-composed emails awaiting human review before sending. Used when `enable_draft_review` setting is `true`.

Column	Type	Nullabl e	Default	Description
<code>id</code>	<code>uuid</code>	NO	<code>uuid_generate_v4()</code>	Primary key
<code>gmail_draft_id</code>	<code>text</code>	YES	—	Unique. Gmail draft ID (set after creating in Gmail API)
<code>session_id</code>	<code>text</code>	YES	—	FK → <code>sessions(session_id)</code> SET NULL on delete
<code>thread_id</code>	<code>text</code>	YES	—	Gmail thread ID (for threaded replies)
<code>to_emails</code>	<code>text[]</code>	NO	<code>' {} '</code>	Recipients
<code>subject</code>	<code>text</code>	NO	<code>''</code>	Subject line
<code>body_text</code>	<code>text</code>	NO	<code>''</code>	Plain text body
<code>body_html</code>	<code>text</code>	YES	—	HTML body
<code>draft_reason</code>	<code>text</code>	YES	—	Why this draft was created (shown in review UI)

<code>status</code>	<code>draft_status</code>	NO	<code>'PENDING_REVIEW'</code>	Review status
<code>reviewed_by</code>	<code>text</code>	YES	—	Email of operator who reviewed
<code>reviewed_at</code>	<code>timestamp tz</code>	YES	—	When reviewed
<code>sent_at</code>	<code>timestamp tz</code>	YES	—	When sent via Gmail API
<code>created_at</code>	<code>timestamp tz</code>	NO	<code>NOW()</code>	—
<code>updated_at</code>	<code>timestamp tz</code>	NO	<code>NOW()</code>	Auto-updated on change

Notes for frontend: Use `v_pending_drafts` view. Show a badge count on the Email Feed nav item when there are `PENDING_REVIEW` drafts. Approve/discard actions call the backend which updates `status`, `reviewed_by`, `reviewed_at`.

9. Booked Meetings

`public.booked_meetings`

Confirmed Google Calendar events. One row per successfully booked meeting.

Column	Type	Nullable	Default	Description
<code>id</code>	<code>uuid</code>	NO	<code>uuid_generate_v4()</code>	Primary key
<code>session_id</code>	<code>text</code>	NO	—	Unique. FK → <code>sessions(session_id)</code>
<code>event_id</code>	<code>text</code>	NO	—	Unique. Google Calendar event ID

<code>event_link</code>	<code>text</code>	NO	—	Google Calendar event URL (from <code>htmlLink</code>)
<code>meet_link</code>	<code>text</code>	YES	—	Google Meet URL
<code>title</code>	<code>text</code>	NO	—	Meeting title (from session)
<code>description</code>	<code>text</code>	YES	—	Event description including AI disclaimer
<code>start_time</code>	<code>timestamp ptz</code>	NO	—	Meeting start (UTC)
<code>end_time</code>	<code>timestamp ptz</code>	NO	—	Meeting end (UTC)
<code>organizer_email</code>	<code>text</code>	NO	—	Google Calendar organizer
<code>attendee_emails</code>	<code>text[]</code>	NO	<code>'{}'</code>	All attendee emails
<code>attendee_statuses</code>	<code>jsonbs</code>	NO	<code>'{}'</code>	RSVP status per attendee (see shape below)
<code>slot_index_used</code>	<code>integer</code>	NO	<code>0</code>	0 = primary slot, 1–2 = fallback
<code>fallback_used</code>	<code>boolean</code>	NO	<code>false</code>	Whether a fallback slot was needed
<code>is_cancelled</code>	<code>boolean</code>	NO	<code>false</code>	Whether this event is cancelled
<code>cancelled_at</code>	<code>timestamp ptz</code>	YES	—	When cancelled
<code>cancellation_reason</code>	<code>text</code>	YES	—	Reason for cancellation
<code>is_rescheduled</code>	<code>boolean</code>	NO	<code>false</code>	Whether replaced by a new booking
<code>rescheduled_to</code>	<code>uuid</code>	YES	—	FK → <code>booked_meetings(</code>

				<code>id</code>) (points to new event)
<code>created_at</code>	<code>timestamp ptz</code>	NO	<code>NOW()</code>	—
<code>updated_at</code>	<code>timestamp ptz</code>	NO	<code>NOW()</code>	Auto-updated on change

JSONB Field Shapes

`attendee_statuses`

```
{
  "alice@example.com": "accepted",
  "bob@example.com": "needsAction",
  "carol@example.com": "tentative"
}
```

Notes for frontend: Use `v_upcoming_meetings` view for the Calendar View. Show `meet_link` as a prominent "Join Meet" button. `slot_index_used > 0` means a fallback was used — display this as a subtle info badge.

10. Meeting Attendees

`public.meeting_attendees`

Normalised per-attendee RSVP tracking. Enables queries like "show all meetings carol@x.com is attending."

Column	Type	Nullable	Default	Description
<code>id</code>	<code>uuid</code>	NO	<code>uuid_generate_v4()</code>	Primary key
<code>meeting_id</code>	<code>uuid</code>	NO	—	FK → <code>booked_meetings(id)</code> CASCADE DELETE
<code>email</code>	<code>text</code>	NO	—	Attendee email address

<code>rsvp_status</code>	<code>attendee_rsvp_status</code>	NO	<code>'needsAction'</code>	Current RSVP status
<code>is_organizer</code>	<code>boolean</code>	NO	<code>false</code>	Whether this attendee is the meeting organizer
<code>updated_at</code>	<code>timestamp_tz</code>	NO	<code>NOW()</code>	Last RSVP status sync

Unique constraint: (`meeting_id`, `email`)

Notes for backend: Sync this table periodically by polling Google Calendar API `events.get()` for RSVP status updates, or when receiving an RSVP notification via Gmail.

11. Session Overrides

`public.session_overrides`

Immutable audit trail of all human override actions. Never updated — append only.

Column	Type	Nullable	Default	Description
<code>id</code>	<code>uuid</code>	NO	<code>uuid_generate_v4()</code>	Primary key
<code>session_id</code>	<code>text</code>	NO	—	FK → <code>sessions(session_id)</code> CASCADE DELETE
<code>override_by</code>	<code>text</code>	NO	—	Email of operator who performed the override
<code>action</code>	<code>override_action</code>	NO	—	<code>CANCEL</code> , <code>RESCHEDULE</code> , or <code>FORCE_BOOK</code>
<code>reason</code>	<code>text</code>	NO	—	Required reason text
<code>new_slot</code>	<code>jsonb</code>	YES	—	New time slot for <code>RESCHEDULE/FORCE_BOOK</code>

<code>previous_status</code>	<code>session_status</code>	NO	—	Snapshot of session status before override
<code>previous_event_id</code>	<code>text</code>	YES	—	Snapshot of calendar event ID before override
<code>applied</code>	<code>boolean</code>	NO	<code>true</code>	Whether override was successfully applied
<code>error_message</code>	<code>text</code>	YES	—	Error if <code>applied = false</code>
<code>created_at</code>	<code>timestampz</code>	NO	<code>NOW()</code>	—

JSONB Field Shape

`new_slot`

```
{ "start": "2025-01-08T10:00:00Z", "end": "2025-01-08T11:00:00Z" }
```

Notes for frontend: Display in the Session Detail audit log. Each entry shows `action`, `reason`, `override_by`, and `created_at`. Read-only — no updates from frontend.

12. Activity Logs

`public.activity_logs`

Append-only full audit trail of every agent and human action in the system.

Column	Type	Nullable	Default	Description
<code>id</code>	<code>uuid</code>	NO	<code>uuid_generate_v4()</code>	Primary key
<code>session_id</code>	<code>text</code>	YES	—	FK → <code>sessions(session_id)</code> SET NULL on delete
<code>email_id</code>	<code>uuid</code>	YES	—	FK → <code>emails(id)</code> SET NULL on delete

<code>event_type</code>	<code>log_event_type</code>	NO	—	Category of this log entry
<code>severity</code>	<code>log_severity</code>	NO	<code>'INFO'</code>	<code>INFO</code> , <code>SUCCESS</code> , <code>WARNING</code> , or <code>ERROR</code>
<code>description</code>	<code>text</code>	NO	—	Human-readable sentence shown in Logs page
<code>payload</code>	<code>jsonb</code>	YES	—	Raw data for the expandable detail section
<code>reasoning_trace</code>	<code>text</code>	YES	—	LangChain agent Thought/Action/Observation chain
<code>actor</code>	<code>text</code>	NO	<code>'AGENT'</code>	<code>'AGENT'</code> , <code>'HUMAN'</code> , or <code>'SYSTEM'</code>
<code>created_at</code>	<code>timestampz</code>	NO	<code>NOW()</code>	—

Notes for frontend: Use `v_activity_logs` view — it includes `meeting_title` and `session_status` from the joined session. Render `severity` as coloured left border. Show `reasoning_trace` in a collapsible `<pre>` block. Filter by `event_type[]` and `severity[]` using Supabase `.in()` filter.

Notes for backend: Write a log entry for every meaningful agent action. Description should be a complete sentence readable by a non-technical operator. `payload` can be any JSON — dump the full API response if useful.

13. Processed Email Hashes

`public.processed_email_hashes`

Idempotency guard. Replaces Redis SETNX. Stores SHA-256 hashes of all processed emails to prevent double-processing.

Column	Type	Nullable	Default	Description
--------	------	----------	---------	-------------

email_hash	text	NO	—	Primary key. SHA-256 of (from_email + ":" + message_id)
first_seen_at	timestamp tz	NO	NOW()	When this hash was first stored
expires_at	timestamp tz	NO	NOW() + 24h	When to auto-purge this record

How to use:

```
-- Check + mark in one atomic call (use this function, not raw queries)
SELECT public.fn_check_and_mark_email('your_sha256_hash_here');
-- Returns TRUE → duplicate, discard this email
-- Returns FALSE → new email, proceed with processing
```

pg_cron cleanup: Runs every hour. `DELETE FROM processed_email_hashes WHERE expires_at < NOW()`

Notes for backend: Always call `fn_check_and_mark_email()` as the very first operation when processing any inbound email, before doing anything else.

14. App Settings

public.app_settings

Global CalSync configuration. Key-value store with type hints for the Settings UI.

Column	Type	Nullable	Default	Description
key	text	NO	—	Primary key. Unique setting identifier
value	jsonb	NO	—	Setting value (typed JSON)
label	text	NO	—	Human-readable label for Settings UI
description	text	YES	—	Help text / tooltip in Settings UI
value_type	text	NO	'string'	'string' 'number' 'boolean' 'text' 'json'

updated_at	timestamp tz	NO	NOW()	—
updated_by	text	YES	—	Email of operator who last changed this

Default Settings

Key	Default Value	Type	Description
calsync_email	"calsync@yourdomain.com"	string	CalSync assistant email address
scheduling_reply_deadline_hours	48	number	Hours before sending availability reminder
max_fallback_slots	3	number	Max alternative slots to try before NO_OVERLAP
max_clarification_attempts	2	number	Max clarification emails per session
ai_disclaimer_text	(see schema)	text	Mandatory AI disclaimer appended to all emails
session_ttl_days	7	number	Days before unresolved sessions expire
enable_draft_review	false	boolean	Require human approval before sending emails

<code>enable_meet_links</code>	<code>true</code>	<code>boolean</code>	Auto-generate Google Meet link on booking
--------------------------------	-------------------	----------------------	---

Notes for frontend: On the Settings page, render each row using `value_type` to choose the input: `boolean` → toggle, `number` → number input, `text` → textarea, `string` → text input. Save via `UPSERT` on `key`.

Notes for backend: Read settings at agent startup and cache them. Re-read when settings are changed (subscribe to Realtime on this table, or poll on each agent run).

15. Views

Pre-built views for common queries. **Always use views over raw table queries where available** — they handle joins and computed columns for you.

`v_sessions_summary`

Used by: Dashboard grid, Session Detail page

Extends `sessions` with:

- `total_participants` — count of all participants
- `replied_count` — how many have submitted availability
- `pending_count` — how many are still waiting
- `participants_replied` — array of replied emails
- `participants_pending` — array of pending emails

Example query (frontend):

```
const { data } = await supabase
  .from('v_sessions_summary')
  .select('*')
  .eq('status', 'AWAITING_REPLIES')
  .order('updated_at', { ascending: false })
  .limit(20);
```

`v_upcoming_meetings`

Used by: Calendar View (upcoming events tab)

Filters to non-cancelled, future meetings. Adds:

- `duration_minutes` — computed meeting duration
- `has_meet_link` — boolean convenience flag

Example query (frontend):

```
const { data } = await supabase
  .from('v_upcoming_meetings')
  .select('*')
  .gte('start_time', new Date().toISOString())
  .lte('start_time', twoWeeksFromNow.toISOString());
```

`v_dashboard_stats`

Used by: Dashboard stat chips (4 counters at top of page)

Returns a single row with:

- `total_active_sessions`
- `awaiting_replies`
- `ready_to_compute`
- `booked_today`
- `needs_attention` (NO_OVERLAP count)
- `currently_computing`

Example query (frontend):

```
const { data } = await supabase
  .from('v_dashboard_stats')
  .select('*')
  .single();
```

`v_email_feed`

Used by: Email Feed page (master list)

Joins `emails` with `sessions` to add:

- `meeting_title` — associated session title
- `session_status` — current session status
- `body_preview` — first 160 chars of body text

v_activity_logs

Used by: Activity Logs page, Session Detail audit log

Joins `activity_logs` with `sessions` to add:

- `meeting_title`
- `session_status`
- `organizer_email`

v_pending_drafts

Used by: Email drafts review panel

Filters `email_drafts` to `status = 'PENDING_REVIEW'`. Adds session context.

16. Functions & Triggers

Functions

`fn_check_and_mark_email(p_hash TEXT) → BOOLEAN`

Purpose: Atomic idempotency check. Call this before processing any inbound email.

```
SELECT public.fn_check_and_mark_email('sha256_hash_string');  
-- Returns TRUE = duplicate → discard  
-- Returns FALSE = new → process
```

`fn_get_session_detail(p_session_id TEXT) → TABLE`

Purpose: Returns a single session with `participants_replied` and `participants_pending` arrays pre-computed.

```
SELECT * FROM public.fn_get_session_detail('sess_a1b2c3d4');
```

Triggers (Automatic — no action needed)

Trigger	Table	Fires On	What It Does
trg_sessions_updated_at	sessions	BEFORE UPDATE	Sets updated_at = NOW()
trg_participants_updated_at	session_participants	BEFORE UPDATE	Sets updated_at = NOW()
trg_prefs_updated_at	participant_preferences	BEFORE UPDATE	Sets updated_at = NOW()
trg_booked_meetings_updated_at	booked_meetings	BEFORE UPDATE	Sets updated_at = NOW()
trg_drafts_updated_at	email_drafts	BEFORE UPDATE	Sets updated_at = NOW()
trg_profiles_updated_at	profiles	BEFORE UPDATE	Sets updated_at = NOW()
trg_log_session_booked	sessions	AFTER UPDATE	Inserts MEETING_BOOKED log when status → BOOKED
trg_log_no_overlap	sessions	AFTER UPDATE	Inserts OVERLAP_COMPUTED WARNING log when status → NO_OVERLAP
trg_create_session_participants	sessions	AFTER INSERT	Creates session_participants rows for each participant

Scheduled Jobs (pg_cron)

Job Name	Schedule	What It Does
<code>cleanup-expired-email-hashes</code>	Every hour	Deletes <code>processed_email_hashes</code> where <code>expires_at < NOW()</code>
<code>expire-stale-sessions</code>	Every 6 hours	Sets <code>status = 'EXPIRED'</code> for sessions past <code>expires_at</code>
<code>purge-old-logs</code>	Weekly (Sunday 2am)	Deletes <code>activity_logs</code> older than 90 days
<code>check-gmail-watch-expiry</code>	Daily (9am)	Logs WARNING if Gmail watch expires within 2 days

17. Who Uses What

Quick reference for each team member. **Use this to know exactly which tables and views you need to touch.**



Frontend Developer

Read these views (not raw tables):

- `v_sessions_summary` → Dashboard grid, Session Detail
- `v_dashboard_stats` → Stat chips on Dashboard
- `v_upcoming_meetings` → Calendar View
- `v_email_feed` → Email Feed page
- `v_activity_logs` → Logs page
- `v_pending_drafts` → Draft review panel

Read these tables directly:

- `participant_preferences` → Preferences page
- `app_settings` → Settings page
- `gmail_watch_subscriptions` → Settings page (watch status)
- `session_overrides` → Session Detail audit log
- `profiles` → Top bar (display name, google_connected)
- `booked_meetings` → Calendar View event detail
- `meeting_attendees` → Attendee RSVP status in Calendar View

Write these tables (via API calls — not direct Supabase writes):

- `participant_preferences` → Preferences form save
- `app_settings` → Settings form save
- `session_overrides` → Override modal (via backend endpoint)

Supabase Realtime — subscribe to:

- `sessions` (all events) → Dashboard live updates
- `emails` (INSERT) → Email Feed live feed
- `activity_logs` (INSERT) → Logs page live feed
- `booked_meetings` (INSERT, UPDATE) → Calendar View live updates

TypeScript types to define (shapes already in the Frontend Requirements doc):

- `Session`, `SessionStatus`, `TimeSlot` from `sessions` + `v_sessions_summary`
- `Email` from `v_email_feed`
- `CalendarEvent` from `v_upcoming_meetings`
- `ParticipantPreferences` from `participant_preferences`
- `ActivityLog` from `v_activity_logs`
- `AppSetting` from `app_settings`

Backend — State/Memory MCP Developer

Your primary tables:

- `sessions` → CREATE, READ, UPDATE status + slots
- `session_participants` → UPDATE `has_replied`, `replied_at`
- `participant_preferences` → READ for VIP weights
- `processed_email_hashes` → use `fn_check_and_mark_email()`

Your primary functions:

- `fn_check_and_mark_email()` → idempotency on every inbound email
- `fn_get_session_detail()` → read session with participant status

Never touch: `booked_meetings`, `oauth_tokens`, `gmail_watch_subscriptions`



Backend — Google Calendar MCP Developer

Your primary tables:

- `booked_meetings` → INSERT on successful booking, UPDATE on cancel/reschedule

- `meeting_attendees` → INSERT all attendees on booking, UPDATE RSVP statuses
- `sessions` → UPDATE `booked_event_id`, `booked_event_link`, `meet_link`, `status`
- `session_overrides` → READ to check for pending overrides
- `oauth_tokens` → READ refresh token for API auth

Never touch: `emails`, `processed_email_hashes`, `gmail_watch_subscriptions`

Backend — Gmail MCP Developer

Your primary tables:

- `emails` → INSERT on every inbound email, UPDATE `processing_status`
- `email_drafts` → INSERT drafts pending review, UPDATE status on send/discard
- `gmail_watch_subscriptions` → UPDATE on watch renewal
- `processed_email_hashes` → use `fn_check_and_mark_email()` before inserting emails
- `oauth_tokens` → READ refresh token for Gmail API auth

Never touch: `booked_meetings`, `meeting_attendees`

Agent Developer (LangChain)

The agent coordinates all MCPs. It does not write directly to Supabase. It calls MCP HTTP endpoints which write to Supabase. For debugging:

Read-only access useful for debugging:

- `v_sessions_summary` → see current state of all sessions
- `v_activity_logs` → trace agent reasoning
- `sessions.collected_slots` → inspect raw slot data

All writes go through MCP APIs — never direct DB writes from the agent.

Appendix — Common Query Patterns

Get all sessions needing attention (NO_OVERLAP)

```
SELECT * FROM public.v_sessions_summary
WHERE status = 'NO_OVERLAP'
```

```
ORDER BY updated_at DESC;
```

Get all participants who haven't replied in a specific session

```
SELECT participant_email  
FROM public.session_participants  
WHERE session_id = 'sess_a1b2c3d4'  
AND has_replied = FALSE;
```

Get full thread email history

```
SELECT * FROM public.emails  
WHERE thread_id = 'thread_abc123'  
ORDER BY received_at ASC;
```

Get all meetings happening this week

```
SELECT * FROM public.v_upcoming_meetings  
WHERE start_time >= DATE_TRUNC('week', NOW())  
AND start_time < DATE_TRUNC('week', NOW()) + INTERVAL '7 days';
```

Get sessions where a specific participant has not replied

```
SELECT s.session_id, s.meeting_title, s.organizer_email  
FROM public.sessions s  
JOIN public.session_participants sp ON s.session_id = sp.session_id  
WHERE sp.participant_email = 'carol@example.com'  
AND sp.has_replied = FALSE  
AND s.status = 'AWAITING_REPLIES';
```

Get activity logs for a session (most recent first)

```
SELECT * FROM public.v_activity_logs  
WHERE session_id = 'sess_a1b2c3d4'  
ORDER BY created_at DESC;
```

Atomic idempotency check (use this, not raw INSERT)

```
SELECT public.fn_check_and_mark_email('abc123def456...');
```

